

Entwicklung eines graphischen Editors für die Audiosignalverarbeitung durch VST-Plugins

Johannes Unger

16. Januar 2012

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation	3
1.2	Der Name	5
2	Anforderungen	6
2.1	Was implementiert werden muss	6
2.1.1	Die Pluginschnittstelle	6
2.1.2	Der Editor	6
2.1.3	Die Plugindatenbank	7
2.1.4	Programmparameter	8
2.1.5	Parameter Verbindungen	8
2.1.6	Zusätzliche Module	8
2.1.7	Persistenz	9
2.2	Was implementiert werden kann	9
2.2.1	Weitere zusätzliche Module	9
2.2.2	Unterstützung alternativer Pluginformate	9
3	Fachliches Umfeld	10
3.1	Der Sampleblock	10
3.2	Der Prozess	10
4	Ausgewählte Problemstellungen	11
4.1	Die Effektverkettung	11
4.2	Das Latenzproblem	12
4.3	Nebenläufigkeit	14
5	Literatur Verzeichnis	20
6	Abbildungs Verzeichnis	21

1 Einleitung

1.1 Motivation

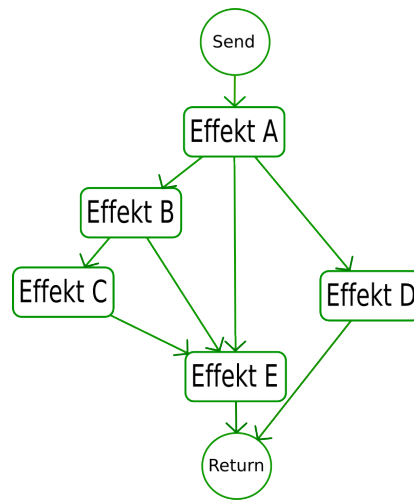


Abbildung 1: Beispiel einer Effektverkettung, wie sie in der Realität möglich wäre

Moderne DAWs¹ bieten umfangreiche und kreative Werkzeuge zum Erzeugen und Manipulieren von Klangereignissen. In vielen Fällen orientierten sich solche Programme anhand real existierender Hardware. So wird oftmals ein Klangereignis einem Kanal in einem virtuellen Mischpult zugeordnet, der einem real existierenden Gerät nachempfunden wurde.

Es können neben der Lautstärkeregelung und Stereoverteilung auch Effekte in einen solchen Kanal eingeschliffen werden. Hier unterscheiden sich allerdings die Möglichkeiten zwischen real existierender Hardware und der Software Lösung. Während es in der Realität unzählige Möglichkeiten gibt verschiedene Effekte zu kombinieren - sprich zu „verdrahten“ (siehe Abbildung 1) - sind die meisten DAWs in ihren Möglichkeiten beschränkt. Dies soll anhand des Sequenzers „Cubase²“ veranschaulicht werden:

Jede Klangquelle in Cubase, sei es eine Audiospur oder ein virtuelles Instrument, wird einem eigenen Kanal in dem virtuellen Mischpult zugeordnet. Jeder Kanal hat neben einem Equalizer und einer Pan-/Lautstärkeregelung eine feste Anzahl von „Insert-Slots“. In einen solchen „Slot“ kann genau ein Effektmodul geladen werden (in Abbildung 2 und 3 als „Effekt A..X“ bezeichnet). Jeder dieser Effekte wird nacheinander durchlaufen, was einer Reihenschaltung entspricht. Für eine Parallelschaltung steht eine begrenzte Anzahl von sogenannten „Send“ Signalwegen zur Verfügung, von denen es zwei Varianten gibt:

¹Digital Audio Workstation: computergestütztes System für Tonaufnahme, Musikproduktion, Abmischung und Mastering

²verwendete Version: Cubase SL 2.0 (<http://www.steinberg.net>)

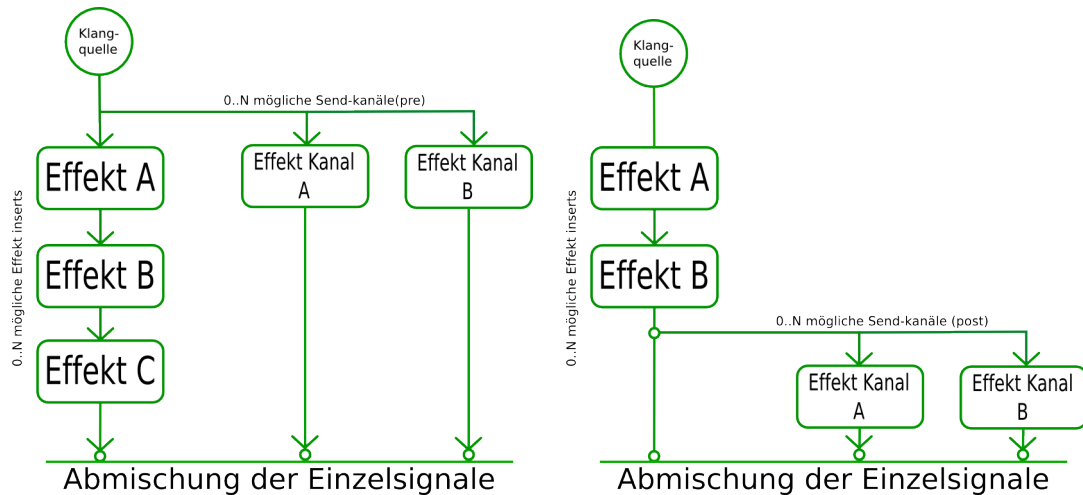


Abbildung 2: Effektkette mit 'send' Abzweigung, jeweils als 'pre' und als 'post' Variante (v.l.n.r)

"pre-send" und "post-send" (siehe Abbildung 2). Je nach Typ wird das Audiosignal vor der Insert-Effektkette abgegriffen oder danach und in einen Effekt Kanal geleitet. Dieser Effektkanal besteht wiederum aus einer begrenzten Anzahl aus "Insert-Slots".

Diese Möglichkeiten sind für die meisten Produktionen vollkommen ausreichend, da der Unterschied - ob ein Effekt in Reihe oder parallel geschaltet wurde - abhängig vom Effekttyp - meistens sehr gering ist. Dennoch ist es manchmal wünschenswert etwas mehr Kontrolle und Übersicht über die Zusammensetzung der einzelnen Signalwege zu haben.

Ziel ist es ein Programm zu entwickeln, das genau das bietet. Es sollte es möglich machen beliebig viele Effekte auf eine virtuelle "Bühne" zu laden um diese wiederum beliebig "verdrahten" zu können. Das Programm sollte selbst ein Effekt sein. Somit wäre die Integration in eine DAW Umgebung denkbar einfach (siehe Abbildung 3).

Die meisten Effekte werden durch Parameter gesteuert. Diese Parameter sollten sich ebenfalls in Form von Reglern auf die "Bühne" holen und von dort aus bedienen lassen. Diese Regler könnte man verbinden und somit mehrere Regler mit einmal steuern. So wäre es zum Beispiel möglich die Intensität eines Halleffektes und die Intensität eines Verzerrers mit einem einzigen Regler gleichzeitig zu steuern. Darüber hinaus bietet es sich an den Signalfluss und das Verhalten der Effekte durch zusätzliche Module manipulierbar beziehungsweise erweiterbar zu machen. Es wäre zum Beispiel eine Art "Step-Sequencer" denkbar. Ein Modul welches Rhythmisch zwischen verschiedenen Signalwegen umschaltet.

Das Programm wäre ein ideales Werkzeug zum erschaffen von kreativen Klangexperimenten.

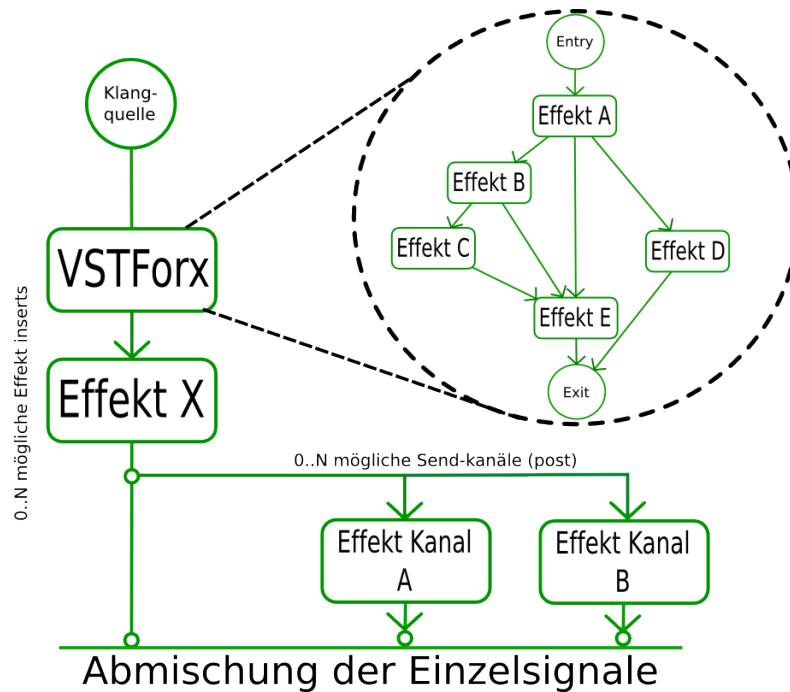


Abbildung 3: Die Idee - freie Verdrahtung als Effekt

1.2 Der Name

Als Programmname entschied ich mich für "VSTForx".

VST VST³ ist eine Plugin-Schnittstelle im Bereich Audioverarbeitung. Aufgrund der hohen Verbreitung von VST-Plugins, wird die zu entwickelnde Software diese Schnittstelle hauptsächlich unterstützen.

Forx ein Kunstwort das vom englischen Wort "fork" abgeleitet ist. Fork bedeutet Gabelung oder Abzweigung und beschreibt somit ziemlich treffend den Hauptzweck der Software.

³Abkürzung für "Virtual Studio Technology" - Steinberg Media Technologies GmbH

2 Anforderungen

2.1 Was implementiert werden muss

2.1.1 Die Pluginschnittstelle

Da VSTForx ein VST-Plugin werden soll muss es dem VST Standard entsprechen. Es stehen hierbei zwei Versionen zur Auswahl: VST2.x und VST3.x.

VST2.x Das VST2.x SDK hatte im Jahre 2006 sein letztes Update erfahren[Stein06]. Seine Ursprünge reichen bis in das Jahr 1990 zurück als die VST Schnittstelle erstmals entwickelt wurde. Da in den 90er Jahren die Rechenleistung begrenzt war, war ein wesentliches Entwurfsziel die Performance. So ist ein Großteil des SDKs in C geschrieben⁴. Aus heutiger Sicht können viele in der VST2.x SDK verwendete Techniken als veraltet betrachtet werden. So wird zum Beispiel zur 'Host nach Plugin Kommunikation' lediglich eine einzige Methode verwendet die über sogenannte Opcodes⁵ parametrisiert werden. Es werden sehr häufig '*void' Zeiger benutzt was zusammen mit einer schlechten Dokumentation schnell zu schwer auffindbaren Fehlern führen kann.

VST3.x Das VST3.x SDK wurde im Jahre 2008 eingeführt und ist eine vollständige Neuentwicklung[Stein08]. Es ist komplett Objektorientiert und bietet eine Menge neuer Funktionen die dabei helfen VST-Plugins schneller und komfortabler zu entwickeln.

Obwohl aus rein technischer Sicht die Entwicklung eines VST3.x-Plugins lukrativer wäre, fiel die Wahl dennoch auf die VST2.x Version. VST2.x-Plugins sind weiter verbreitet und werden von fast allen DAWs unterstützt.

Ein VST-Plugin tritt in der Form einer Dynamisch ladbaren Bibliothek auf. Je nach System ist es unter Windows eine ".dll" Datei oder unter Mac OSX ein sogenanntes "Resource Bundle".

2.1.2 Der Editor

Der Editor ist die Graphische Schnittstelle zu VSTForx. Er sollte folgende Dinge beinhalten:

Die Bühne Die Bühne ist die Hauptaktionsebene und sollte somit den größten Teil des Editorfensters ausmachen. Die Effektverkettung wird hier erstellt, dargestellt und kann von hier aus auch verändert werden. Plugin Parameter lassen sich als Regler platzieren und bedienen. Ausserdem sollte es von der Bühne aus möglich sein die Editoren der geladenen Plugins zu öffnen und zu benutzen.

⁴es existieren aber - um die Implementierung zu vereinfachen - C++ Wrapperklassen.

⁵Abkürzung für Operation Code. Jede aufrufbare Operation wird durch einen Code Symbolisiert und als Parameter übergeben.

Die Toolbox Um die Bühnenfunktionalität umzusetzen bedarf es einer Unterscheidung von verschiedenen Mausaktionen. So ist es ein Unterschied ob man zum Beispiel einen Regler dreht oder ihn verschiebt. Die Mausaktion ist aber in beiden Fällen das "ziehen"⁶. Es werden also verschiedene Mausekontexte benötigt. Diese lassen sich in der Toolbox in Form von Knöpfen auswählen.

Das Menü Über ein Menü sollten folgende Funktionen erreichbar sein:

- hinzufügen und entfernen von Bühnenobjekten
- manipulation bzw. anpassen von Bühnenobjekten

Da die meisten Menüfunktionen abhängig vom jeweiligen Bühnenobjekt sind, ist es empfehlenswert das Menü in Form eines Kontextmenüs zu implementieren. Ausserdem sollte es zur Übersichtlichkeit möglich sein das Menü in Unterpunkte aufzuteilen.

Der Setupdialog Von hier aus lassen sich VST-Pluginverzeichnisse festlegen und der "Scanvorgang" (siehe 2.1.3) kann von hier aus gestartet werden. Es sollte von hier aus auch möglich sein die Editorfenstergröße zu bestimmen.

2.1.3 Die Plugindatenbank

Um ein Plugin öffnen zu können würde es im einfachsten Fall ausreichen das Plugin mittels eines "Datei öffnen" Dialoges zu lokalisieren. Dies würde aber schnell umständlich da mit jedem zu öffnenden Plugin durch eine Ordnerstruktur navigiert werden müsste, die zum Großteil aus (Plugin-) uninteressanten Informationen besteht. Ausserdem kann - zumindest unter Windows - nicht zwischen einer System ".dll" Datei und einer VST-Plugin Datei unterschieden werden. Hier bestünde die "Gefahr" eine Datei öffnen zu wollen die gar kein VST-Plugin ist.

Ein weiteres Problem ist es dass VSTForx in der Lage sein soll den kompletten Bühneninhalt zu speichern und zu einem späteren Zeitpunkt wieder zu laden. Würde nun in solch einer Speicherdatei auf absolute Pfade verwiesen, bestünde die Gefahr dass zu einem späteren Zeitpunkt sich die Ordnerstruktur⁷ geändert hat und der gesicherte Pfad somit ungültig wäre.

Sicherer wäre es eine Datenbank zu haben die alle ladbaren Plugins enthält und zur Verfügung stellt. Durch einen einmaligen "Scanvorgang" würde die bestehende Ordnerstruktur durchlaufen und alle gefundenen Plugindateien würden der Datenbank hinzugefügt werden. Sollte sich einmal die Ordnerstruktur ändern müsste nur der "Scanvorgang" wiederholt werden. Für Speicherdateien wäre das Plugin weiterhin verfügbar.

⁶engl.: "Drag"

⁷oder die Speicherdatei wird auf einem anderen Gerät geöffnet

2.1.4 Programmparameter

Alle Objekte die in VSTForx durch Parameter steuerbar sind sollten ihre Parameter als Regler für die "Bühne" verfügbar machen. Diese Regler sind verbindbar.

Ausserdem sollte das Programm eine Menge an (VST-)Parametern bieten. Diese lassen sich ebenfalls auf der "Bühne" als Regler ablegen. Somit ist es möglich alles "regelbare" in VSTForx von der DAW aus zu steuern.

2.1.5 Parameter Verbindungen

Um noch mehr Möglichkeiten zu haben wäre es sinnvoll in eine Parameterverbindungen Operatoren, die das Verhalten einer Verbindung beeinflussen, einfügen zu können. Wäre zum Beispiel Regler A mit Regler B verbunden, würde bei einem "Inverse" Operator, Regler B zudrehen wenn Regler A aufgedreht würde.

Mögliche Operatoren wären:

- Inverse
- Exponential/Logarithmisch ⁸
- Logarithmisch/Exponential
- Offset (ein fester Verschiebungswert)

2.1.6 Zusätzliche Module

Zusätzliche Module könnten als "interne Effekte" bezeichnet werden. Sie können wie Plugins in den Signalweg eingebunden werden und haben die Möglichkeit diesen zu manipulieren. Genau wie Plugins besitzen Module eine Menge an Parametern durch die Modulspezifische Einstellungen möglich sind.

Mögliche Module:

- VolumeNode - ermöglicht die Lautstärkeregelung vom Signalweg
- PanNode - ermöglicht die Stereoverteilung vom Signalweg
- Schalter - ermöglichen das umschalten beliebig vieler Eingänge/Ausgänge.
- Stepsequencer - ermöglichen das automatische rhythmische Umschalten beliebig vieler Eingänge/Ausgänge.

⁸eine Operation hat zwei Richtungen ("A zu B" und "B zu A") wobei "B zu A" die Inverse Operation von "A zu B" sein muss.

2.1.7 Persistenz

Das Programm sollte in der Lage alle Einstellungen und Konstruktionen dauerhaft zu speichern. Dabei ist zu beachten dass - wird VSTForx in einem DAW Projekt benutzt - VSTForx Bestandteil dieses Projektes ist. Wird also ein solches Projekt gespeichert, sollten mit dem erneuten öffnen des Projektes, alle vorherigen VSTForx Konstruktionen vollständig wiederhergestellt werden.

2.2 Was implementiert werden kann

2.2.1 Weitere zusätzliche Module

- Peak2Value - transformiert den Ausschlag (engl. Peak) eines Signals in ein Parameterwert. Solch ein Parameterwert wird als (nicht veränderbarer) Regler dargestellt und kann mit anderen Reglern verbunden werden.
- ADSR2Value - wird an einem Signalpfad ein Schwellwert überschritten, startet dieses Modul eine ADSR⁹ Sequenz. Wie schon beim "Peak2Value" Modul wird der Wert dieser Sequenz in einen Parameterwert transformiert.
- Midi2Value - transformiert MIDI-ereignisse wie Modulation oder Pitch in einen Parameterwert.

2.2.2 Unterstützung alternativer Pluginformate

Denkbar wäre es weitere bekannte (Audio-) Pluginformate zu unterstützen. Sowohl in der Form dass VSTForx alternative Pluginformate kennt und öffnen kann, als auch dass VSTForx in einem alternativen Pluginformat in compilierbar ist.

Alternative Pluginformate wären:

- VST3.x
- AudioUnit (Mac OSX exclusives Pluginformat)
- DirectX Plugin
- LV2 (ist eine OpenSource Audio-Plugin Variante und Weiterentwicklung von LAD-SPA¹⁰)

⁹ADSR - Attack, Decay, Sustain, Release: idealisierten Phasen der Hüllkurve eines Klanges

¹⁰Abkürzung für: Linux Audio Developers Simple Plugin API

3 Fachliches Umfeld

3.1 Der Sampleblock

Ein digitales Audiosignal besteht aus einer Menge einzelner Werte, den Samples. Der Sampleblock stellt die Menge an Samples dar, die von einem VST-Plugin auf einmal bearbeitet werden kann. Die Größe des Sampleblockes ist ausschlaggebend für die Performance und die Latenz¹¹ einer DAW Umgebung. Je niedriger die Blockgröße desto niedriger die Latenz, desto höher die CPU Auslastung.

3.2 Der Prozess

In der Terminologie des VST-SDK, ist mit Prozess der Teil eines Plugins gemeint, der für die Bearbeitung aller Pluginein- und ausgänge verantwortlich ist.

Im VST2.x SDK ist dies eine Methode namens "processReplacing". Ihr werden als Parameter die Eingangs- und Ausgangssampleblöcke übergeben. Die Anzahl der Sampleblöcke ist abhängig von der Anzahl der Pluginein- sowie Ausgangskanäle. So werden zum Beispiel einem Stereoplugin zwei Eingangssampleblöcke übergeben, jeweils ein Block für "Rechts" und ein Block für "Links".

Der Ablauf eines Prozesses kann wie folgt skizziert werden:

- die DAW füllt die Plugin-Eingangssampleblöcke mit den Daten eines Audiokanals
- die DAW ruft die "processReplacing" Methode des Plugins auf.
- das Plugin verarbeitet die Eingangssampleblöcke und schreibt die Ergebnisse in die entsprechenden Ausgangssampleblöcke
- die DAW verarbeitet die Ausgangssampleblöcke weiter

Dieser Vorgang wird, während eine DAW im "Abspielmodus", ist ständig wiederholt.

¹¹die Verzögerung zwischen Signalein- und ausgabe.

4 Ausgewählte Problemstellungen

4.1 Die Effektverkettung

Das Modell VSTForx soll Plugins laden und mittels "virtueller" Kabel verbinden können. Als mathematisches Modell ist für diese Aufgabe der gerichtete Graph geeignet. Die Knoten repräsentieren die Plugins und die Kanten ihre Verbindungen. Eine gerichtete Kante entspricht einer Verbindung von Pluginausgang zu Plugineingang.

Die Berechnungsreihenfolge Wie bereits in Kapitel 3.2 beschrieben, erzeugt ein Plugin seine Ausgabe anhand einer "processReplace" Methode. Das Programm muss nun den beschriebenen Prozessablauf mit den geladenen Plugins durchführen. Doch in welcher Reihenfolge sollten diese Aufrufe stattfinden?

Gegeben sei folgendes Beispiel:

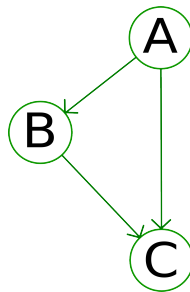


Abbildung 4: Beispiel eines Plugin-Graphen

Der erforderliche Prozessablauf für Abbildung 4 wäre:

1. bereite Inputsampleblöcke für Plugin-A vor
2. führe "processReplace" mit Plugin-A aus
3. bereite Inputsampleblöcke für Plugin-B vor (mit der Ausgabe von Plugin-A)
4. führe "processReplace" mit Plugin-B aus
5. bereite Inputsampleblöcke für Plugin-C vor (mit der Ausgabe von Plugin-A und B)
6. führe "processReplace" mit Plugin-C aus
7. übertrage die Ausgabe von Plugin-C in die Programm(VSTForx) Ausgabesampleblöcke

Wie in Schritt 5 zu sehen, ist zur korrekten Ausführung von Plugin-C notwendig, dass die Plugins A und C bereits bearbeitet wurden. Das bedeutet: Plugin-C ist von Plugin-A und Plugin-B abhängig.

In der Graphentheorie gibt es einen Algorithmus der in der Lage ist Abhängigkeiten aufzulösen: Die Topologische Sortierung.

Die Topologische Sortierung basiert auf der Durchlaufstrategie Tiefensuche¹².

"Sie erzeugt eine lineare Anordnung von Knoten eines Graphen. Existiert eine Kante zwischen den Knoten (u, v) , so wird u vor v geordnet."¹³

Das Resultat der Topologischen Sortierung ist daher, eine sortierte Menge an Knoten, deren Reihenfolge genau ihren Abhängigkeiten entspricht. In unsrem Beispiel würde die Topologische Sortierung die gewünschte (Plugin-)Knotenmenge $\{A, B, C\}$ ¹⁴ ergeben.

Die Topologische Sortierung hat ein Laufzeitverhalten von $\mathcal{O}(|V| + |E|)$ ^{13 15}, und ist somit eher ungeeignet um in Echtzeit sehr häufig ausgeführt zu werden. Allerdings ändert sich die Ergebnismenge auch nur dann, wenn sich der Graph verändert. Die Ergebnismenge sollte daher ausschließlich nach einer Graphänderung erstellt und in einer Liste gespeichert werden. Solch eine "Prozess-Liste" wird während des Prozesses einfach iteriert. Somit ergibt sich eine Laufzeit von $\mathcal{O}(|V|)$.

Rückkopplungen Eine Einschränkung der Topologischen Sortierung ist es, dass die zu sortierenden Graphen keine Zykel enthalten dürfen. Dies hat für VSTForx zur Auswirkung, dass keine Rückkopplungen "geschaltet" werden können. Eine Rückkopplung wäre es zum Beispiel, wenn der Ausgang eines Plugins, direkt oder über Umwege, wieder an seinem Eingang liegen würde. Sollten Rückkopplungsschaltungen unterstützt werden, könnte man die betreffenden Knoten lokalisieren und durch einen Subgraphen substituieren. Dieser Subgraph könnte dann die Knoten durch einen alternativen Algorithmus, einen der Rückkopplungen unterstützt, bearbeiten. Dies ist jedoch, aufgrund der Komplexität des Problem, nicht vorgesehen.

Die Topologische Sortierung ist allerdings in der Lage zu erkennen ob ein Graph Zyklisch ist¹³. Zur Vermeidung von Fehlverhalten werden daher Zyklische Graphen nicht zugelassen.

4.2 Das Latenzproblem

Werden Plugins parallel verbunden, kann es unter Umständen zu einem Problem kommen: es gibt Plugins die eine Latenzzeit größer 0 aufweisen. Das bedeutet die Pluginausgabe ist zeitlich verzögert. Bei einer Parallelschaltung kann diese Latenz nun bewirken, dass

¹²oder auch DFS - engl.: Depth First Search

¹³vergl. [BGL01], S. 176 f.

¹⁴vorausgesetzt Knoten A ist der Startknoten der Topologischen Sortierung

¹⁵wobei $|V|$ die Menge aller Knoten und $|E|$ die Menge aller Kanten ist

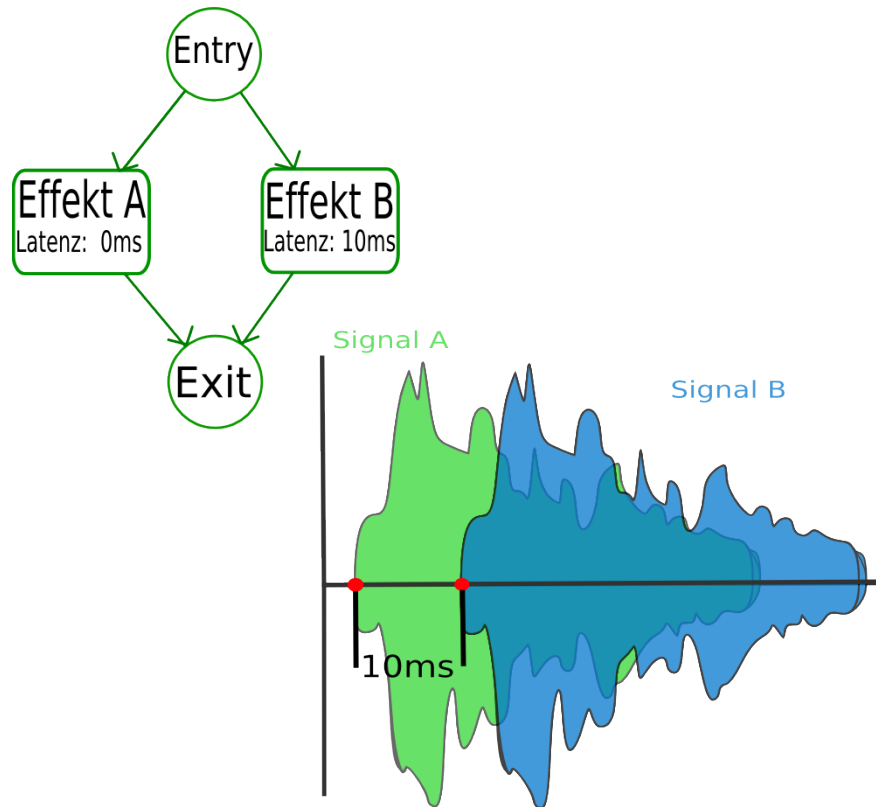


Abbildung 5: Darstellung des Latenzproblems: die Signalausgabe aus "Effekt B" ist um 10 ms verzögert

zum Beispiel in einem Signalpfad eine Verzögerung auftritt, im anderen aber nicht (siehe Abbildung 5). Die Signale erklingen daher versetzt.

Um unterschiedliche Latenzen in einem komplexen Graph auszugleichen, wäre folgendes notwendig:

- hinzufügen eines berechneten Latenzwert je Knoten
= Maximaler Latenzwert der Vorgängerknoten, addiert zum eigenen (realen) Latenzwert
- hinzufügen eines Buffers je Knoten dessen Größe = Sampleblockgröße + berechneter Latenzwert ist.
- die Ausgaben aller Vorgängerknoten, je nach Vorgängerlatenz an unterschiedlichen Position, dem Buffer hinzufügen (siehe Abbildung 6)
- den Buffer als Eingabe weiterverarbeiten

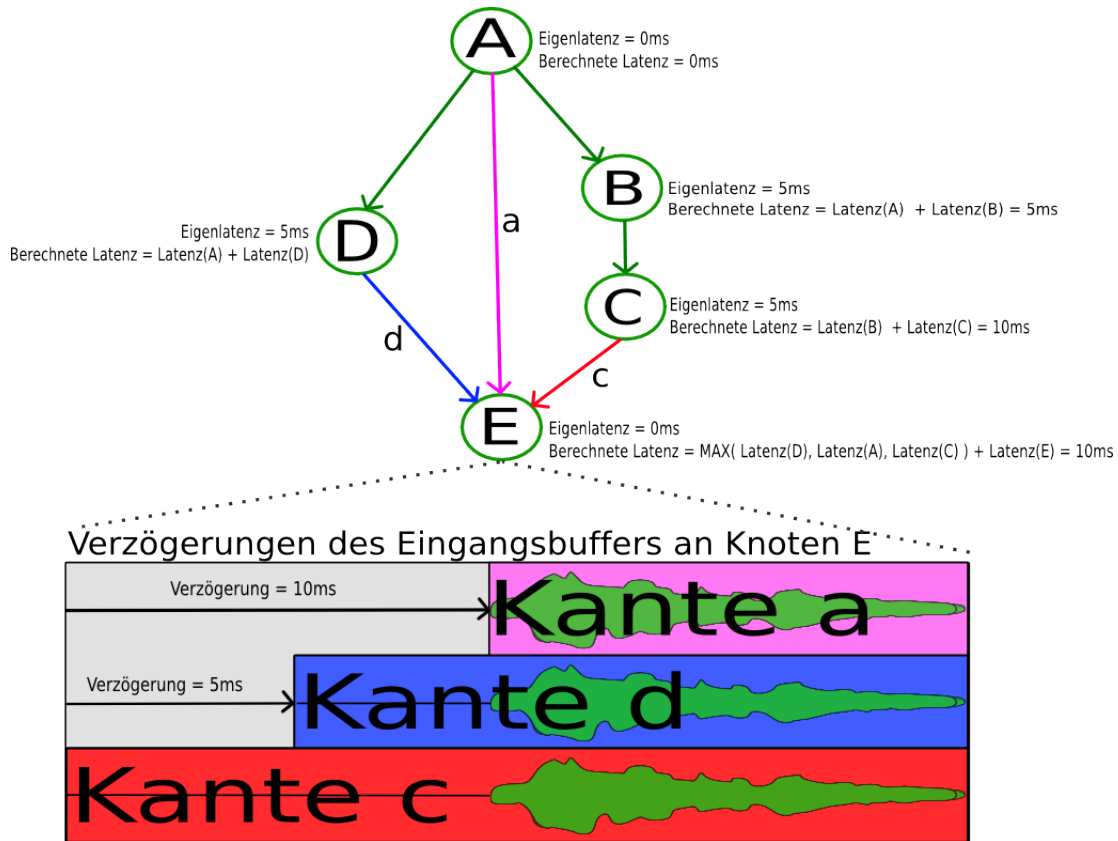


Abbildung 6: Ausgleich unterschiedlicher Latenzen in einem komplexen Graph

4.3 Nebenläufigkeit

Ein VST-Plugin kann optional einen Editor implementieren. Ein Editor ist ein eigenständiges Fenster, welches die Benutzeroberfläche des Plugins enthält und wird von der DAW bei Bedarf geöffnet. Da Eingaben auf der Benutzeroberfläche zu einem unbestimmten Zeitpunkt geschehen, der Pluginprozess aber kontinuierlich (nebenläufig¹⁶) abgearbeitet wird, liegt hier ein Nebenläufigkeitsproblem vor. Wird der Effekt-Graph geändert, während er vom Prozess bearbeitet wird, kann dies ein undefiniertes Verhalten verursachen. Folgend sollen drei verschiedene Lösungsansätze beschrieben werden:

Blockieren des Prozesses Es wäre möglich jede Graphverändernde Methode solange zu blockieren, wie die Prozessmethode abgearbeitet wird. Umgekehrt wird die Abarbeitung der Prozessmethode blockiert, solange eine Graphverändernde Methode abgearbeitet wird. Ein Verfahren welches solch eine Blockierung unterstützt ist das Mutex (siehe [TAN09]).

¹⁶ob als eigenständiger Prozess oder Thread ist DAW abhängig

Listing 1: Beispiel einer gegenseitigen Blockierung

```
class Graph {
private:
    Mutex mutex;
public:
    void veraenderGraph() {
        mutex.lock();
        // ...
        mutex.unlock();
    }

    void prozess() {
        mutex.lock();
        // ...
        mutex.unlock();
    }
};
```

Der Nachteil dieser Methode ist, je komplexer die Graphklasse und ihre enthaltenden Methoden, so höher ist die Gefahr dass es vergessen wird ein gesperrtes Mutex wieder zu entsperren (siehe Listing 2). Ein Deadlock wäre die folge.

Listing 2: Beispiel eines potentiellen Deadlocks

```
class Graph {
private:
    Mutex mutex;
public:
    void veraenderGraph() {
        mutex.lock();
        if (dings == bums) {
            return; // !!
        }
        mutex.unlock();
    }

    void prozess() {
        mutex.lock();
        // ...
        mutex.unlock();
    }
};
```

```

    }
};

```

Das Befehl Entwurfsmuster Das Befehl Entwurfsmuster gehört zu den Verhaltensmustern der "GoF" Entwurfsmuster. Es dient dazu Programmbefehle in einem Objekt zu kapseln (vgl. [GOF04]).

Der Graph würde seine verändernden Befehle nicht mehr als öffentliche Methoden anbieten, sondern nur noch Befehlsobjekte entgegennehmen. Solch ein Objekt würde dann vom Graph, zu einem sicheren Zeitpunkt (zum Beispiel nach Abschluss der Prozessmethode) ausgeführt (siehe Listing 3).

Listing 3: Lösung des Nebenläufigkeitsproblem durch das Befehl Entwurfsmuster

```

// Datei AenderGraph.h

#include "ICommand.h"
#include "Graph.h"

class AenderGraph : public ICommand {
friend class Graph;
private:
    Graph *graph;
public:
    AenderGraph( Graph *graph ) : graph(graph) {
    }

    virtual void execute() {
        graph->veraenderGraph();
    }
};

// Datei Graph.h
#include "ICommand.h"

class Graph {
private:
    void veraenderGraph() {
        // ...
    }
    typedef stack<ICommand*> Commands;
    Commands commands;
public:
    void addCommand( ICommand *cmd ) {

```



```

        commands.push(cmd);
    }
    void prozess() {
        // bearbeite Graph
        // ...
        while ( !commands.empty() ) {
            ICommand *cmd = commands.top();
            cmd->execute();
            commands.pop();
        }
    }
};

```

Da es möglich ist ein VST-Plugin über die DAW in einen Standbymodus¹⁷ zu schalten, kann es mit diesem Verfahren passieren, dass die entgegengenommenen Befehlsobjekte nicht mehr ausgeführt werden. Der Graph würde nicht mehr bearbeitet. Daher ist diese Vorgehensweise nicht geeignet.

Automatisches Blockieren und Freigeben Mittels Konstruktor/Destruktor Mechanismen ist es möglich eine Lösung zu entwerfen, die dafür sorgt, dass der Prozess automatisch blockiert und wieder freigegeben wird solange der Graph verändert wird. Realisiert wird dies durch eine Klasse die "Janitor"¹⁸ genannt wurde. Die Janitor-Klasse enthält alle Graph verändernden Methoden. Der Konstruktor des Janitors blockiert die Prozessmethode, der Destruktor gibt sie wieder frei. Somit kann sichergestellt werden, dass solange ein Janitor-Objekt existiert der Graph nicht weiter abgearbeitet werden kann (siehe Listing 4).

Zusammen mit einem "Shared Pointer" Mechanismus¹⁹ und dem Singleton Entwurfsmuster (siehe [GOF04]) kann man dafür sorgen, dass ein Janitor-Objekt, von mehreren Stellen aus, gleichzeitig benutzt werden kann.

Da nach jeder Graphänderung eine Prozess-Liste (siehe 4.1) erstellt werden muss, existiert mit dieser Lösung ein weiterer Vorteil: Die Prozesslisten Erstellung könnte ebenfalls mit dem Destruktor des Janitors abgearbeitet werden. Somit würde Automatisch die Prozessliste nach jeder Änderung angepasst werden.

Listing 4: Lösung des Nebenläufigkeitsproblemes durch den Janitor

¹⁷die Prozess Methode wird nicht mehr ausgeführt

¹⁸engl.: Hausmeister

¹⁹Shared Pointer sind Mechanismen die dafür sorgen dass allozierte Objekte automatisch freigegeben werden, sobald sie nicht mehr verwendet werden (vgl. [?]).

```
// Datei Janitor.h
```

```
class Janitor {
private:
    Graph *graph;
public:
    Janitor( Graph *graph ) : graph(graph) {
        // Graph-Prozess sperren
        graph->mutex.lock();
    }

    virtual ~Janitor() {
        // Prozessliste erstellen
        // ...
        // Graph-Prozess entsperren
        graph->mutex.unlock();
    }

    void veraenderGraph() {
        graph->veraenderGraph();
    }
};
```

```
// Datei Graph.h
```

```
class Graph {
friend class Janitor;
public:
private:
    Mutex mutex;
    void veraenderGraph() {
        // ...
    }
public:
    void prozess() {
        // bearbeite Graph
    }
};
```

```
// Datei GUIController.cpp
```

```
void GUIController::veraenderGraph() {

    Janitor janitor ( getGraph() ); // konstruktor sperrt Graph-Prozess
```

```
    janitor->veraenderGraph();  
    // ...  
    // ...  
}  
// janitor verlaesst Gueltigkeitsbereich  
// und entsperrt Graph-Prozess
```

5 Literatur Verzeichnis

Literatur

- [Stein06] Steinberg Media Technologies GmbH. *Steinberg releases VST 2.4 standard with new features*. <http://www.steinberg.net/index.php?id=334&L=1>, 2006
- [Stein08] Steinberg Media Technologies GmbH. *Steinberg releases VST3 SDK*. http://www.steinberg.net/en/company/press/archive/2008/steinberg_vst3_sdk.html, 2008
- [BGL01] Addison-Wesely. *The Boost Graph Libraray*.
- [GOF04] Addison-Wesely. *Entwurfsmuster*
- [TAN09] Andrew S. Tanenbaum: *Moderne Betriebssysteme*. Pearson Education 2009, ISBN 9783827373427, S. 176ff

6 Abbildungs Verzeichnis

Abbildungsverzeichnis

1	Beispiel einer Effektverkettung, wie sie in der Realität möglich wäre	3
2	Effektkette mit 'send' Abzweigung, jeweils als 'pre' und als 'post' Variante (v.l.n.r)	4
3	Die Idee - freie Verdrahtung als Effekt	5
4	Beispiel eines Plugin-Graphen	11
5	Darstellung des Latenzproblem: die Signalausgabe aus "Effekt B" ist um 10 ms verzögert	13
6	Ausgleich unterschiedlicher Latenzen in einem komplexen Graph	14